

Aula 09: Strings em Python

Principais funções e formatação avançada

Maurício da Silva Pires
mspires@id.uff.br

TCC00326 - Programação de computadores
Niterói, Rio de Janeiro

Revisitando as aulas anteriores

- ⌋ Nas aulas anteriores, trabalhamos com os conceitos de subprogramação e listas
 - * Construímos algumas funções iterativas e recursivas em Python
 - * Aprendemos sobre listas lineares, que são estruturas de dados indexadas em que os elementos são dispostos sequencialmente
- ⌋ Na Aula 05, ao falar sobre os tipos de dados em Python, vimos alguns exemplos básicos de manipulação de *strings*
- ⌋ Nesta aula, vamos explorar as semelhanças entre as *strings* e as listas em Python, já que ambos implementam ideias de indexação

O que falamos na Aula 05

- Objetos da classe **str** denotam textos
 - Os literais do tipo texto sempre são colocados entre aspas duplas ou aspas simples
- Os caracteres de uma *string* (texto) são numerados, da esquerda para direita de 0 a $n - 1$, onde n é o total de letras
 - Índices negativos de -1 a $-n$ fazem acesso da direita para a esquerda
 - $p[i]$ indica o caracter de índice i da *string* p
- A função **len(p)** retorna o número de caracteres da *string* p
- Aprendemos a formatar *strings* com o método **format**

Interpolação de strings

- ⌋ A interpolação de *strings* em Python permite inserir valores de variáveis e expressões diretamente dentro de um texto
 - * O primeiro contato com a interpolação foi feito usando o método **format**
 - * Agora vamos trabalhar de uma forma mais moderna, as *f-strings*

```
from math import pi
```

```
print("{} é o valor de pi".format(pi))
```

```
# 3.141592653589793 é o valor de pi
```

```
print("{} é o valor de pi".format(round(pi,4)))
```

```
# 3.1416 é o valor de pi
```

- ⌋ As **f-strings** (ou *formatted string literals*) são a forma mais moderna, rápida e legível de formatar e interpolar *strings* no Python (disponíveis desde a versão 3.6)
 - * Basta adicionar a letra f (ou F) antes das aspas e colocar variáveis ou expressões diretamente entre chaves
 - * Podemos adicionar até mesmo formatações nos textos a partir dessas expressões

```
def saudar(nome: str, tipo="Olá": str) -> None:
    print(f"{tipo}, {nome}!")
    return None
```

```
def zeros_antes(valor: int) -> None:
    print(f"{valor:08}") # imprimir um inteiro com 8
    # caracteres com 0s à esquerda
    return None
```

- Podemos restringir o número de casas decimais de um número real a partir da adição de `:.xf` após a variável na interpolação, onde x é um inteiro não-negativo
 - Quando $x = 0$, o número real é truncado e é exibida apenas a parte inteira
 - A notação `:0x.yf` imprime o número com x caracteres no total, sendo y caracteres após o ponto e com $x - (y + 1)$ caracteres na parte inteira, preenchendo com zeros

```
from math import pi
```

```
print(f"{pi:.2f}")           # 3.14
print(f"{pi:.5f}")           # 3.14159
print(f"{pi:08.3f}")          # 0003.142
```

Contornando a imutabilidade

- ⌋ Uma forma simples de alterarmos uma *string* é por meio de uma operação de *casting* para o tipo lista
 - * Usamos a função **list(P)** para transformar a *string* P em uma lista de caracteres (*strings* de comprimento 1)
 - * Na lista resultante, alteramos os caracteres que queremos
 - * Após as alterações, precisamos reconstituir a *string* usando o método **join** sendo chamado a partir de uma *string* vazia
- ⌋ O método **join** compõe uma *string* a partir de um iterável de *strings* e usando a *string* que chamou o método como separador
 - * A sintaxe é **separador.join(iterável)** onde o argumento é um iterável de *strings* (**obrigatoriamente**)

Exemplo

```
# Transformar string em strong
```

```
palavra = "string"  
lista = list(palavra)      # ["s","t","r","i","n","g"]  
lista[3] = "o"            # ["s","t","r","o","n","g"]  
palavra = "".join(lista)  # strong
```

```
# Compondo coisas com join
```

```
# O iterável precisa ser um iterável de strings
```

```
L = ["lua","de","mel"]
```

```
sep1 = ""                # string vazia  
sep2 = " "  
sep3 = "-"
```

```
print(sep1.join(L))      # luademel  
print(sep2.join(L))      # lua de mel  
print(sep3.join(L))      # lua-de-mel
```

Separando strings

☾ Podemos dividir uma *string* em uma lista de *substrings*

- * O método **split(separador)** é chamado a partir de uma *string* e retorna a lista
- * É um método interessante para separarmos dados que são lidos, por exemplo, em uma única linha
- * A lista resultante é uma lista de *strings*, portanto pode ser necessário converter cada um dos trechos para outro tipo de dado

```
strg1 = "10 23 34 45 20 30"  
lista = strg1.split()  
# lista = ["10", "23", "34", "45", "20", "30"]
```

Exemplo

```
# Adicione o método no módulo turmaProg
def lista_str_to_int(lista: list[str]) -> list[int]:
    saida = []
    for elemento in lista:
        saida.append(int(elemento))
    return saida

if __name__ == "__main__" :
    strg1 = "10 23 34 45 20 30"
    lista = strg1.split()
    # lista = ["10","23","34","45","20","30"]

    lista = lista_str_to_int(lista)
    # lista = [10,23,34,45,20,30]

    # Alternativa pythônica
    # lista = list(map(int,strg1.split()))
    # o método map aplica o método int a cada
    # um dos elementos da coleção strg1.split()
```

Verificação de números em string

- ☾ O método **.isdecimal()** é um método utilizado para verificar se o texto é composto exclusivamente por caracteres numéricos decimais (dígitos de 0 a 9)
 - * Ele retorna **True** se todos os caracteres da *string* forem dígitos decimais e houver pelo menos um caractere; caso contrário, retorna **False**
 - * Rejeita os algarismos árabes ocidentais ou indianos, além de expoentes, frações e algarismos romanos
- ☾ Trabalhamos com cenários em que os literais numéricos são do seguinte tipo
 - * Inteiros começam com “-”, “+” ou nenhum símbolo e o resto são caracteres de 0 a 9
 - * Reais vale a ideia dos inteiros e um único “.” para marcar a diferença entre a parte decimal e a parte inteira

Exemplo

```
# Revisando o método anterior
def lista_str_to_int(lista:list[str]) -> list :
    saida = []
    for elemento in lista:
        if elemento.isdecimal():
            saida.append(int(elemento))
        elif (
            (elemento[0] == "-" or elemento == "+")
            and elemento[1:].isdecimal()
        ) :
            saida.append(int(elemento))
        else : return [None]
    return saida

if __name__ == "__main__" :
    strg1 = "10 23 34 45 20 30"
    lista = strg1.split()
    lista = lista_str_to_int(lista)
    # o método map pode gerar um erro caso alguma
    # string em questão não seja decimal
```

Exercício

- ⌋ Se baseando na função **lista_str_to_int** para criar uma função que converta uma lista de *strings* em uma lista de números reais
- * Todo literal de número real possui apenas números decimais e um único ":" (Na prática, poderia ser uma vírgula, mas nossa convenção é só com ponto)
 - * Estamos assumindo que não é colocado um espaço em branco entre o sinal e o número
 - * Os números podem começar com os sinais "+", "-" ou então sinal omitido para o caso positivo
 - * Em caso de erro, retorne a lista [None]
 - * Lembre-se que estamos mantendo a consistência com o retorno da função

Exercício

```
def lista_str_to_float(lista: list[str]) -> list :
    saida = []
    for elemento in lista:

        if elemento.isdecimal():
            saida.append(float(elemento))

        elif ( (elemento[0] == "-" or elemento == "+")
              and elemento[1:].count(".") == 1
              and elemento[1:].replace(".", "").isdecimal
              () ) : saida.append(float(elemento))

        elif ( elemento.count(".") == 1
              and elemento.replace(".", "").isdecimal()
              ): saida.append(float(elemento))

        else : return [None]

    return saida
```

Semelhanças entre listas e strings

- ⌈ A menos da mutabilidade, listas lineares e *strings* possuem características e métodos semelhantes, já que ambos são iteráveis ordenados de maneira linear
- ⌈ Métodos e operadores compartilhados, por exemplo
 - * **.count(x)**: Conta quantas vezes o elemento x aparece
 - * **.index(x)**: Retorna o índice da primeira ocorrência de x e um erro se não encontrar
 - * **in / not in**: Verifica se um elemento existe na sequência
 - * **Fatiamento**: Retorna subsequências
 - * Concatenação (+) e repetição (*): Junta sequências ou as multiplica

Método `replace`

- ⌋ O método **`replace`**(S_1, S_2) tem um comportamento semelhante à decomposição que fizemos.
 - * Gera uma nova *string* na qual **TODA** ocorrência de uma *substring* S_1 é substituída por uma *string* S_2

```
palavra = "babaca"

nova = palavra.replace("ca", "do")

print(palavra)          # babaca
print(nova)             # babado

nova = palavra.replace("ba", "bo")

print(nova)             # boboca
```

Método *find*

- ☾ O método **find**(*S*,*ini*,*fim*) retorna o índice de onde começa a primeira ocorrência da *string* *S* a partir da posição *ini* até a posição *fim*
- * Por padrão, *ini* = 0
 - * Por padrão, *fim* = *len*(*S*)
 - * Retorna -1 quando não encontra ocorrência de *S* no intervalo especificado

```
palavra = "azul doce azul"
palavra.find("azul")           # 0
palavra.find("azul",2)         # 10
palavra.find("azul",2,8)       # -1
```

Mais formatação de string

- ⌋ O método **.strip()** é utilizado para remover espaços em branco (ou outros caracteres específicos) do início e do fim de uma *string*
 - * Sem argumentos, ele remove espaços, tabulações (`\t`) e quebras de linha (`\n`) das extremidades
 - * Você pode passar uma *string* com quais caracteres deseja remover das pontas
 - * O Python vai apagar qualquer combinação deles até encontrar um caractere diferente
- ⌋ Podemos variar a remoção de um lado específico, usando as versões **.lstrip()** para o início e **.rstrip()** para o final

Exemplo

```
# Remove os caracteres 'x', 'z' e ',' das extremidades
url = "xxx,://site.com,zzz"
print(url.strip("xz,"))  # Saída: "://site.com"

# Remove pontos do início e do fim
codigo = "...12345..."
print(codigo.strip("."))  # Saída: "12345"

texto = "    Python    "
print(f"Esquerda limpa: -{texto.lstrip()}-")
# Saída: -Python -
print(f"Direita limpa:  -{texto.rstrip()}-")
# Saída: -    Python-
```

Como o `.strip()` só limpa as pontas, se você precisar
remover espaços duplicados no meio do texto, a
melhor técnica é combinar `.split()` com `.join()`

Normalizando strings

- Quando buscamos uma *substring* em uma *string*, vamos precisar garantir uma previsibilidade de como está o texto
 - Python é **case sensitive**, ou seja, diferencia maiúsculas de minúsculas
 - Normalizar** a *string* significa jogar tudo para maiúsculo ou minúsculo e remover os acentos e cedilhas

```
# Funções .upper e .lower
```

```
texto = "Uma Análise dO tExTo"
```

```
texto.lower()           # uma análise do texto
```

```
texto.upper()           # UMA ANÁLISE DO TEXTO
```

Normalizando strings

- ⌈ A maneira mais eficiente e nativa de remover acentos e cedilhas é usar a função *normalize* da biblioteca padrão **unicodedata** combinada com as funções *encode* e *decode*
 - * Ela decompõe os caracteres acentuados (como “ç” ou “ã”) em duas partes (base + acento) e ignora os sinais gráficos ao converter para ASCII'
 - * **normalize(forma,texto)** e precisaremos usar a forma NFKD (Forma de Normalização Unicode de Decomposição de Compatibilidade) decompõe caracteres visualmente ou semanticamente semelhantes em seus elementos básicos
 - * A forma NFKD separa uma letra com acento ou outro símbolo em dois itens separados
 - * As funções **encode(cod,erros)** e **decode(cod,erros)** recebem os tipos de codificações e os tipos de erros a tratar (o tipo de codificação padrão do Python é ASCII)

Exemplo

```
# Remoção de acentos e símbolos especiais
from unicodedata import normalize

# Adicione esta função e a importação
# ao módulo turmaProg

def remove_diacritics(texto: str) -> str:
    norm = normalize("NFKD", texto)
    norm = norm.encode("ASCII", "ignore")
    return norm.decode("ASCII")

if __name__ == "__main__" :
    strg1 = "Não SabéMõs cÕmÔ as Coisás ficARÃO"
    strg1 = remove_diacritics(strg1)
    print(strg1)
    # NaO SabeMos cOmO as Coisas ficARAO

# Após a remoção, basta colocar o texto todo
# em maiúsculas ou todo em minúsculas
```

